



Skylar 2: Parameter-Parity Hybridisation of Recurrent and Attention Layers for Domain- Specialised Code Models

A. Ivanovitch

skyl4r.ai

Technical report · 31 July 2026
github.com/skyl4r-ai/skylar

On the status of this report. Published as transparent research for the Skylar framework, not as a general-purpose or SOTA model. The Skylar 2 architecture is in the public framework, behind flags; **no Skylar 2 model has been trained at scale yet**. Every number below is either measured on our own hardware and reproducible from the repository, or cited to a primary source. Where our measurements contradict a source we cite, we report both and say which regime applies. Section 7 lists what we tried and **rejected**, including one component we implemented, measured, and removed.

ABSTRACT

We describe **Skylar 2**, a revision of the Skylar decoder-only architecture aimed at domain-specialised code models — specifically COBOL, a language whose programs and copybooks make long-context capability a functional requirement rather than a benchmark line. The revision hybridises Kimi Delta Attention (KDA) with full softmax attention at a 3:1 ratio, adds Attention Residuals (AttnRes), replaces SwiGLU with a bounded SiTU-GLU variant, and gates the attention output.

Our two methodological contributions are small but consequential. First, a **parameter-parity dimensioning rule** for hybridisation: because linear-attention layers have no grouped-query compression, transplanting a source model's head count inflates the parameter budget by 10-16%; equating the two layer types instead yields $H_{kda} = (n_{heads} + n_{kv_heads})/2$, which is integral across our whole preset family and brings the cost of hybridisation to **+1.0-1.8%**. Second, an **algebraic reformulation of AttnRes** that removes its dominant memory cost: since $RMSNorm(v) \cdot (w \otimes q) = (v \cdot wq) / rms(v)$, the depth-attention scores reduce to two matrix-vector products producing scalars, eliminating the materialisation of stacked residual sources and cutting activation memory **9.5×** at bit-level equivalence with the reference implementation.

On a controlled three-seed A/B over 6,450 real COBOL programs at 112M parameters, the revised architecture reaches **21.9% lower perplexity** ($6.487 \rightarrow 5.066$, paired $t = 20.4$, $p \approx 0.002$) at **+3.0% parameters**, while training **2.45× faster** at sequence length 4096. Total architectural overhead, after two components were measured out, is **+1.0-1.9%** depending on preset.

We report two negative results as first-class findings. Multi-token prediction, implemented and measured under identical conditions, showed **no benefit** (best validation cross-entropy identical to four decimal places) at -11% throughput, and was removed. And the per-channel output gate — the form used by the architecture we drew from — is **statistically indistinguishable** over five seeds from a per-head variant costing **127× fewer parameters**, which we adopted instead.

1. Introduction

1.1 What Skylar is

Skylar is a from-scratch decoder-only training framework written in plain PyTorch with no abstraction layers, scaling from a $\sim 6\text{M}$ test preset to 128B. The v1 architecture (Section 2) follows the Qwen3 lineage: RMSNorm [1], RoPE [2], grouped-query attention [3], QK-Norm, SwiGLU [4], with FlexAttention document masking and optional μP [5].

A constraint that shapes everything below: Skylar models are trained **entirely from random initialisation**. We never warm-start from third-party weights and never distil from a closed model into a released checkpoint. This is a hard project rule, and it is why architectural efficiency matters to us more than it would to a group that can fine-tune an existing base: every improvement in sample efficiency is an improvement we can actually spend.

1.2 Why COBOL, and why long context is not optional

Legacy banking, insurance and public-administration systems run on COBOL; the developers who maintain them are retiring; and general-purpose code models saw comparatively little COBOL during pre-training. A small, local, sovereign specialist can plausibly beat generalists on the niche.

This target has an architectural consequence. A COBOL program is not self-contained: it is a program plus its copybooks plus its JCL plus record layouts, and correctness depends on relations spanning all of them. A model that can only attend over a short window is not a weaker COBOL model — it is one that cannot express the task. **Long context is a functional requirement here, and it is what motivates hybridisation.**

Our published v1 specialist, `Skylar-980M-Cobol`, illustrates both the promise and the ceiling. On COBOLEval — 146 HumanEval-derived COBOL problems evaluated by GnuCOBOL compilation and execution — it reaches **80.1% compile-success rate**, above the 73.95% of COBOL-Coder-14B, a model $14\times$ larger and fine-tuned from an existing base rather than trained from scratch. But its **pass@1 is 7.5** against their 49.3. Repeated post-training variants (CoT, DPO/ORPO, instruct, conversational) each *degraded* COBOLEval further,

which we read as a capacity bound rather than a data problem: the model can be taught to produce compilable COBOL, but not to produce *correct* COBOL, and no amount of post-training moved that.

Skylar 2 is our attempt to attack that bound from the architecture side before spending it on scale.

1.3 Contributions

1. **Parameter-parity hybridisation rule** (§3.2): $H_{kda} = (n_{heads} + n_{kv_heads})/2$, which makes recurrent/attention hybridisation approximately free in parameters at any width, and is integral across our preset family.
2. **Memory-free AttnRes** (§4.2): an algebraic reformulation reducing activation memory $9.5\times$ at $1.6e-06$ agreement with the reference kernel, restoring the full-depth form as a design choice rather than a memory concession.
3. **A document-isolation failure mode specific to hybrids** (§6.1): we show that the short convolution preceding the recurrence leaks across packed-document boundaries, and — critically — that the contamination is *not* bounded by the convolution width but propagates through the entire subsequent document via the recurrent state.
4. **A decode-throughput analysis** (§6.3) showing that the widely-repeated claim that hybrids decode more slowly at short context is, at our scales, an artefact of launch-bound generation loops rather than a property of the architecture.
5. **A reproducible empirical comparison** (§8) with three seeds, two explicit negative results — one of which removes $127\times$ of parameter cost from a component we had adopted by imitation — and a stated list of limitations (§9).

2. Background: the v1 architecture

NanoTransformer is a HuggingFace-native PreTrainedModel. Per layer:

```
x → RMSNorm → GQA attention (QK-Norm, RoPE) → residual
      → RMSNorm → SwiGLU FFN → residual
```

Head dimension is decoupled from d_{model} / n_{heads} (Qwen3-style), so Q/O projections may be rectangular. Three attention paths are selected automatically: FlexAttention with a block mask for packed training, dense SDPA as fallback, and causal SDPA for generation.

The presets relevant here, at vocabulary 64,000 with tied embeddings:

preset	d_model	L	H / kv	d_head	d_ff	parameters
1B_D	1536	36	12/4	128	4096	1,004,395,008
2b (new, §5)	2048	36	16/4	128	7168	2,094,164,992
4b	2560	36	32/8	128	9728	3,797,351,936

All parameter counts in this report are computed arithmetically and verified against the instantiated model; the discrepancy is zero on every preset (`utils/bin.arch_budget.py`).

3. Hybridising recurrence with attention

3.1 Why a hybrid rather than a replacement

KDA [6, 7] maintains a fixed-size matrix state updated by a delta rule with a per-channel forget gate:

$$S_t = (I - \beta_t k_t k_t^\top) \cdot \text{Diag}(\alpha_t) \cdot S_{t-1} + \beta_t k_t v_t^\top$$

The state is *finite*: it compresses history rather than retaining it. That is precisely the trade we want for most layers — cost per token independent of context — but it is the wrong trade for *exact recall*, which COBOL needs (a `PIC` clause declared in `WORKING-STORAGE` constrains a `MOVE` hundreds of lines later, verbatim). Following Kimi Linear [6], we therefore keep a 3:1 ratio: 27 recurrent layers to 9 full-attention layers, with the last layer always full attention so that the representation feeding the LM head has exact recall available.

3.2 The parameter-parity rule

KDA has no grouped-query compression: its `q`, `k`, `v`, `o` projections are all $d \times \text{key_dim}$. Our attention layers with GQA compress `k` and `v` by the group factor. Consequently **adopting the source model's head count inflates the model**: with `H_kda = d/128` as in Kimi's configuration, hybridisation costs +15.95% parameters on our 990M preset and +14.27% on the 2B.

Equating the two layer types removes the inflation. Attention costs $2 \cdot d \cdot d_h \cdot (H + kv)$ and KDA costs $4 \cdot d \cdot d_h \cdot H_{\text{kda}}$; setting them equal:

$$H_{\text{kda}} = \frac{n_{\text{heads}} + n_{\text{kv heads}}}{2}$$

The rule is integral on every preset in our family, which we take as weak evidence that it is not a forced fit:

preset	attn/layer	H_kda	key_dim	KDA/layer	Δ /layer	model Δ	recurrent state
990M (12/4)	6.29M	8	1024	6.97M	+0.68M	+1.83%	512 KB
2B (16/4)	10.49M	10	1280	11.37M	+0.89M	+1.15%	640 KB
4B (32/8)	26.21M	20	2560	27.61M	+1.40M	+0.99%	1280 KB

The residual 1-2% is KDA's fixed overhead — short convolution, `b_proj`, low-rank decay and gate — which scales with d rather than d^2 . Hybridisation therefore becomes *cheaper* as the model grows.

Two dependent choices. **The output gate is kept low-rank** ($d \rightarrow 128 \rightarrow \text{key_dim}$, as in Kimi Linear) rather than full-rank as in K3: full-rank would force $H_{kda} = 2(H+kv)/5$ to stay at parity — 6 heads instead of 8 on the 990M — halving the recurrent state, which is the layer's associative memory. We prefer to spend on state and economise on the gate. **The state is also the honest limitation:** 512 KB/sequence against a KV cache of ~ 400 MB at 32k tokens is the advantage, but 27 of 36 layers have *finite* memory, and the 9 attention layers are load-bearing, not a compatibility detail.

3.3 FLOPs

Hybridisation reduces whole-model forward FLOPs at $T=8192$ by **10.9%** (990M), **4.7%** (2B) and **13.5%** (4B). As Section 6.3 shows, this does not translate into proportional decode speedup at batch 1, for reasons that have nothing to do with FLOPs.

4. Attention Residuals

4.1 The mechanism

In a standard transformer every block reads one tensor — the residual stream — which is the unweighted sum of everything before it. AttnRes [8] replaces the sum with a choice: the stream is zeroed at each block boundary, previous block outputs remain available as separate sources, and each application point weights them by a softmax against a learned pseudo-query:

$$k_1 = \text{RMSNorm}(v_1) \quad p = \text{softmax}_1(k_1 \cdot q) \quad o = \sum_1 p_1 v_1$$

Cost is $2d$ parameters per application point — 224k in total on the 990M, **0.02%** of the model. This is what makes it interesting for a capacity-bound model: expressivity that does not come from size.

We initialise q to zero, so the softmax starts uniform and AttnRes begins as the *mean* of available sources — a neutral, stable starting point that the model then learns to skew. Random initialisation would start from an arbitrary routing that the run carries as noise throughout.

4.2 Removing the memory cost

The reference implementation stacks the sources: `torch.stack` of L tensors of shape $[B \cdot T, D]$. On the 990M at $B \cdot T = 65\,536$ this is **13.7 GiB** of live activations, which is why practical deployments fall back to short windows.

It is unnecessary. Since $\text{RMSNorm}(v) \cdot (w \odot q) = (v \cdot wq) / \text{rms}(v)$, each source's score is computable from **two reductions over D that produce scalars**:

$$\text{score}_1 = \frac{v_1 \cdot wq}{\text{rms}(v_1)} \quad wq = w \odot q \text{ (precalcolato una volta)}$$

Written as matrix-vector products rather than elementwise products followed by a sum, no $[B \cdot T, D]$ intermediate is ever materialised. Measured on an RTX 4090, full-depth form, 73 sources, $B \cdot T = 8192$, bf16:

	activation memory
stacking (reference form)	17,593 MiB
this formulation	1,848 MiB

9.5× less, and the residual is dominated by the gradients of the sources themselves, which any implementation must produce. Agreement with `fla`'s reference `naive_attnres` is **1.6e-06** relative.

The practical consequence is that the choice between a short window ($S = 4-6$) and the full-depth form returns to being about expressivity rather than about HBM.

5. Bounded activations, gating, and the intermediate preset

5.1 SiTU-GLU

SwiGLU multiplies two unbounded factors, so the activation scale is not known a priori. SiTU-GLU soft-clips both:

$$\text{situ}(x) = \beta_1 \tanh(W_1 x / \beta_1) \odot \sigma(W_1 x) \odot \beta_2 \tanh(W_3 x / \beta_2)$$

For small activations $\tanh(z) \approx z$ and the function **coincides with SwiGLU** (measured deviation $1.2\text{e-}05$ at $\sigma = 0.05$); for large ones it saturates, and the output magnitude is bounded by $\beta_1 \beta_2 = 100$ (measured: exactly 100.000 on extreme inputs). A known activation range is the precondition for FP8/FP4 quantisation without retraining, which is why it enters now rather than later: it cannot be retrofitted to a trained model.

A measurement that overturned our own prior estimate. Our earlier analysis put SiTU-GLU at +21-24% time and +33% memory, and on that basis recommended deferring it. That measurement was of the *unfused* form. With the elementwise core under `torch.compile` (RTX 4090, `1B_D` FFN, $B \cdot T = 8192$, bf16):

	ms/iter	peak memory
SwiGLU, compiled	6.88	404 MiB
SiTU-GLU, compiled	6.91	468 MiB
SwiGLU, eager	7.28	520 MiB
SiTU-GLU, fused kernel, eager	6.90	456 MiB

+0.6% time, not +21-24%. Memory grows 15.8%, not 33%.

One implementation note with a real failure mode: the fused kernel must be compiled with `dynamic=True`. With `dynamic=False`, inductor specialises on sequence length; during

generation the length increases by one per token, and after eight recompilations PyTorch silently falls back to eager — removing the fusion precisely at inference, with only a warning.

⚠ $\beta_1 = 4$, $\beta_2 = 25$ are K3's values, tuned for their regime (hidden 7168). For us they are a starting point to re-tune, **not a founded choice**.

5.2 Output gate

Gated attention [9] applies a sigmoid gate to the attention output before the output projection, reported to add non-linearity and sparsity and to remove the attention sink. We compute the gate from the **layer input** rather than the attention output, so its decision does not depend on what attention has already produced, and apply an RMSNorm to the attention output first, following K3's formulation:

$$y = W_o [\sigma(W_g x) \odot \text{RMSNorm}(\tilde{o})]$$

We apply it only to the 9 full-attention layers. The 27 KDA layers already carry a native output gate as part of the delta-rule formulation, so adding one there would be redundant; restricting it to the attention layers costs a quarter as much, and the per-head form measured in §8.4 reduces it further to +0.017%.

The variant question is open in the literature we could verify: [9]'s abstract specifies a "head-specific sigmoid gate" over 30 tested variants without fixing whether the gate is one scalar per head (`Linear(d, H)`) or one value per output channel (`Linear(d, H·d_head)`). The two differ by **127×** in parameters (18,560 vs 2,359,424 on the 990M). K3 uses the per-channel form; we measured both and adopted the per-head form (§8.4).

5.3 The 2b preset

Skylar 2 introduces an intermediate preset to validate that the pipeline scales before committing to the 4B. Constraints: `d_head=128` fixed as in the whole family; $H \cdot d_h = d_{\text{model}}$ (square W_q , as in 1B_D); GQA 4:1, between the 3:1 of 1B_D and the 4:1 of 4b; $d_{\text{ff}}/d_{\text{model}} = 3.50$, monotone between 2.67 and 3.80; aspect ratio L/d intermediate; $d_{\text{ff}} = 7168 = 56 \times 128$, tile-aligned. `H_kda = 10` follows from §3.2 and is integral. The preset was derived independently by two procedures that agreed to the byte.

6. Implementation: three failure modes worth publishing

Each of these produces a *worse model without an error*, which is the class of bug that costs the most and is reported the least.

6.1 Document isolation in hybrids is not what it looks like

In packed-sequence training, attention layers are isolated by a block mask. **A recurrent layer has no mask to modify — it has a state**, and the only way to separate documents is to tell the kernel where to reset it (`cu_seqlens`). This much is known.

What we did not find stated anywhere, and what our tests forced us to discover: **the short convolution preceding the recurrence leaks too, and its leak is not bounded by the kernel width**. A causal depthwise convolution of width 4 lets three tokens at a document boundary see the previous document — but those three tokens then enter the recurrence, and the contamination propagates through the *entire* subsequent document.

Measured, on an isolated KDA layer, comparing a document read alone against the same document read after another:

	max deviation per position after the boundary
cu_seqlens, convolution unmasked	0.848, 0.699, 0.652, 0.609, 0.375, 0.262, 0.387, 0.432
no convolution (conv_size=1), cu_seqlens	0.000 × 8 — exact
cu_seqlens + masked convolution	0.000, 0.004, 0.005, 0.007, 0.005, 0.006, 0.005, 0.005

The middle row shows the kernel isolates perfectly; the first shows the convolution defeating it at *every* position, not just the first three. Masking the convolution taps at segment starts restores isolation (exact in fp64 against two independently-run convolutions; **106×** separation in bf16 on the full model).

None of this raises an error. A hybrid trained without it simply learns worse.

6.2 Name-based initialisation

Skylar applies depth-scaled initialisation $\mathcal{N}(0, 0.02/\sqrt{(2L)})$ to residual output projections, matched **by parameter name** (`W_o.weight`, `w2.weight`). Any new module whose output projection is named otherwise — `o_proj`, the conventional name in most codebases — **silently skips it**. We named KDA's output projection `W_o` for this reason and added an automated coverage gate; naming a module correctly is not a style question when initialisation is name-dispatched.

A related trap in the same class: our gradient checkpointing passed block arguments **positionally**. Adding `cu_seqlens` to the block signature silently routed it into `use_cache`. The fix is to bind by keyword through a closure.

6.3 Decode is launch-bound, and this explains the "hybrids are slower" folklore

Hybrid architectures are commonly reported as decoding more slowly than attention at short context. We reproduce that observation and then show, at our scales, that it is not a property of the architecture.

Decode at the 4B preset (3.93B parameters, bf16, RTX 4090), separating the time Python spends *enqueueing* kernels from total wall time:

	enqueue (CPU)	total	CPU share
v1 (attention)	15.32ms	15.36ms	100%
v2 (hybrid KDA)	23.65ms	23.68ms	100%

The terminal `synchronize()` adds **0.04 ms out of 15.36**. The GPU has finished and is idle; time per token *is* launch time. KDA issues roughly twice as many kernels per layer as attention, and that is the entire gap.

The prediction follows: if launches dominate, widening the batch should be nearly free. Measured at context 512:

batch	v1 tok/s	v2 tok/s	v2/v1
1	65.4	42.7	0.65×
8	390.6	356.2	0.91×
32	710.6	1064.4	1.50×

From batch 1 to 32 the attention model scales 10.9×; the hybrid scales **24.9×**, and overtakes. At that point KDA's real advantages — fewer FLOPs and a 4× smaller cache — become visible, because the bottleneck has moved to the GPU where they apply.

The same reversal occurs along context length at fixed batch 1, with the crossover at **~6-7k tokens** at 4B (against ~48k at 112M — the crossover moves down with scale, since per-layer work grows relative to launch overhead):

context	v1 tok/s	v2 tok/s	v2/v1
256	65.3	43.2	0.66×
4,096	55.0	45.3	0.82×
16,384	23.5	45.6	1.94×

Note the hybrid's column: 43, 45, 46, 46 — **flat**. Cost per token independent of context is the property being purchased. The attention baseline falls 65 → 23.

A correction to our own earlier reporting. We previously cited 11× at 32k and an 819× smaller state, taken from single-layer microbenchmarks. Those figures do not survive contact with a full 3:1 hybrid: in the hybrid, 9 of 36 layers retain a KV cache, so the cache is **4×** smaller, not 819×, and the speedup at 32k at batch 1 is below unity. We report the corrected figures and note the discrepancy rather than the more favourable ones.

7. What we rejected, and why

7.1 Multi-token prediction — implemented, measured, removed

MTP [10] predicts two tokens ahead through an auxiliary head fusing the trunk hidden state with the embedding of the incoming token. We implemented it with chunked cross-entropy (a second head over a 64k vocabulary otherwise doubles the logits tensor: 4.19 GB in bf16 at batch 4×8192) and measured it under conditions identical to §8, comparing **the principal head's cross-entropy** rather than the total loss, which with MTP includes the auxiliary term:

step	without MTP	with MTP
100	3.6507	3.6850
300	2.1672	2.1697
500	1.6435	1.6435
599	1.6744	1.6840

Best validation identical to four decimal places; intermediate checkpoints marginally *worse*. Cost: **-11% throughput, +1.2 GiB, +3% parameters. Removed.**

Honest scoping: the public evidence for MTP is DeepSeek-V3 [10], a 671B MoE trained far longer. It is possible MTP helps at scale and that a 112M/600-step probe cannot see it. But we have no evidence it helps *us*, we have measured cost, and the payoff that would justify it — speculative decoding — is not implemented in our serving stack. The code remains behind a default-off flag.

7.2 NoPE — implemented, disabled for the first run

K3 removes positional encoding entirely, letting the recurrence supply position, which yields extrapolation beyond the trained window without YaRN. We implemented per-layer RoPE bypass and then disabled it, for a reason specific to our data pipeline rather than to the idea.

Our loader draws a **random offset inside a document**. With NoPE, the recurrent layers become the model's *only* source of positional information — so we would be training that sole mechanism on states that begin mid-program. And NoPE is not retro-fittable: a model trained without RoPE cannot receive it afterwards. The failure would be invisible in the loss and visible only in a long-context retrieval evaluation, i.e. after the run.

The flag exists; the first run keeps RoPE on the 9 attention layers, at zero parameter cost.

7.3 Muon — deferred, with a specific reason

Per-head Muon is attractive and costs no parameters. We defer it for one precise reason: our learning rate is not searched but **derived by ratio** from a known-good AdamW run (§8.1). Muon operates on a different learning-rate scale; transferring AdamW hyperparameters requires the update-RMS rescaling of Moonlight [11], and without it the LR is off by roughly two orders of magnitude. That failure does not crash — it descends slowly and looks stable.

Eight architectural modifications produce a model that is either working or broken, and cheap gates can distinguish those. An out-of-regime learning rate produces a model that is **plausible and worse**, which is the case we cannot diagnose. Muon therefore enters at the next scale, where a baseline exists against which to search its LR.

8. Experiments

8.1 Setup

Data. 6,450 real COBOL programs (10.8M tokens) drawn from our corpus — GitHub mainframe repositories, AWS mainframe modernisation samples, defect suites, teaching material — tokenised with the published Skylar-980M-Cobol BPE (vocab 48,128). Real programs, not synthetic.

Models. medium preset (112M). **A** = v1 architecture. **B** = v2: KDA 3:1 + AttnRes (S=4) + output gate + SiTU-GLU. Identical data, seed, learning rate, schedule, batch size and document masking in both arms.

Protocol. 600 steps, seq 1024, batch 8, LR 3e-4 cosine, warmup 60, dropout 0. Selection and comparison on validation cross-entropy of the principal head. Three seeds.

8.2 Quality

seed	v1	v2	Δ
1234	1.8896	1.6435	-0.2461
777	1.8642	1.6372	-0.2270
2024	1.8558	1.5869	-0.2689
mean	1.8699	1.6225	-0.2473

Three of three, same sign. The effect is **12× the between-seed standard deviation** (0.0171). Paired $t = 20.4$ on 2 degrees of freedom, $p \approx 0.002$. Perplexity **6.487 → 5.066: 21.9% lower**.

The effect is not size. Saved checkpoints are 112.48M (v1) and 115.84M (v2): **+3.0%**. No scaling law converts a 3% parameter increase into a 22% perplexity reduction; the gain is architectural.

8.3 Throughput and memory

Training, at parity (document masking active in both arms):

seq_len	v1 tok/s	v2 tok/s	v2/v1	v1 GiB	v2 GiB
1024	22,934	35,624	1.55×	7.34	8.51
2048	12,513	27,671	2.21×	9.78	10.95
4096	7,181	17,592	2.45×	14.66	15.82

The advantage grows with sequence length, as it must — dense attention is quadratic, the recurrence is linear. Document masking also weighs asymmetrically: FlexAttention's block mask runs on every layer in v1 and on 9 of 36 in v2, the remainder using `cu_seqlens`, which is far cheaper. The same 600 steps took 206 s against 290 s.

8.4 Output gate variant: the 127× question

Since [9] does not settle whether the gate is per-head or per-channel, we measured both against no gate at all, five seeds, everything else identical:

seed	per-head	per-channel	Δ
1234	1.6381	1.6499	−0.0118
777	1.6190	1.6372	−0.0182
2024	1.6064	1.5869	+0.0194
31337	1.5196	1.4980	+0.0216
8888	1.4831	1.4686	+0.0145
mean	1.5732	1.5681	+0.0051 ± 0.0084

Indistinguishable: $t = 0.61$ on 4 degrees of freedom, and the sign of the difference **reverses** across seeds — two favour per-head, three favour per-channel. There is no evidence that the per-channel form's **127× larger parameter cost** (2,359,424 vs 18,560 per layer at 1B_D) purchases anything.

We adopt the per-head gate. When two options are statistically indistinguishable, the cheaper one wins. This reduces the gate's contribution from **+2.11% to +0.017%** of model parameters, and brings total v2 overhead to **+1.87% / +1.18% / +1.02%** across the three presets.

This measurement also illustrates why we ran five seeds rather than two. At two seeds the difference was −0.0150 in favour of per-head and we recorded it internally as a weak signal; it was noise, and the three additional seeds reversed it. The between-seed spread in this experiment (up to 0.066 for the ungated arm) exceeds the effect being measured, which is exactly the regime in which small- n comparisons mislead. We note this because the same trap applies to COBOLEval: at $n = 146$ problems, the binomial standard error on compile-success rate near 0.80 is 3.3 points, so differences below roughly 9 points are not distinguishable from noise without a paired protocol.

8.5 Cross-vocabulary evaluation

Perplexity is **not comparable across vocabularies**: cross-entropy measures bits per *token*, and a 64k-vocabulary token covers more bytes on average than a 48k one. Since Skylar 2 moves from 48,128 to 64,000, we established a **bits-per-byte** baseline on a frozen byte set before starting, using the published `Skylar-980M-Cobol`:

slice	bpb	bytes/token
cobol_real	0.2985	3.188
cobol_synth	0.3206	2.873
code_general	1.0022	3.397
it_legal	1.6481	3.149
mean	0.8174	

The set is frozen and versioned; regenerating it would invalidate every past comparison. The profile also characterises v1 honestly: a COBOL specialist (0.30) that is weak on Italian legal prose (1.65), consistent with a code-dominated mixture.

9. Limitations

We state these because a report of this kind is only useful if its boundaries are explicit.

1. **Scale.** All comparative results are at 112M over 4.9M tokens. The target is 4B over 300B — a 36× difference in parameters and roughly 60,000× in tokens. The evidence supports "the architecture is sound and better where we can measure it", **not** a quantitative prediction at 4B.
2. **Shared learning rate.** Both arms ran at 3e-4, optimal for neither. Correcting this requires a separate LR search per architecture, which we have not done. Our expectation is that a re-tuned LR widens rather than narrows the gap — architectures with more gating typically tolerate higher LR — but this is an expectation, not a result.
3. **Single domain, single validation source.** Validation is drawn from the same COBOL corpus as training. We have not measured generalisation to other languages or to prose.
4. **Decode measurements are for a naive Python generation loop.** They characterise our current serving path, not the achievable ceiling; CUDA graphs would remove the launch overhead that dominates §6.3.
5. **Learning-rate transfer.** Peak LR (below) are derived by ratio from a single known-good run rather than searched. They are an extrapolation, and instability at constant LR would be corrected by lowering the peak first.
6. **β_1 , β_2 for SiTU-GLU are inherited, not tuned** (§5.1).
7. **The 4B decode figures are for an untrained model:** they measure throughput, which is weight-independent, but no quality claim at 4B is made or implied.

Schedule and learning rate

We adopt WSM [12] — constant LR with post-hoc merging of the final checkpoints, no online decay — because it permits selecting the merge window on an executable benchmark rather than on loss. Peak LR follows the scaling law of [13], applied **as a ratio** from our known-good point (980M at 3e-4 over 20.37B tokens) rather than in absolute terms, since the paper's constants come from a different codebase:

$$\text{LR}(N, D) = 3 \times 10^{-4} \cdot (N / 0.98 \times 10^9)^{-0.2219} \cdot (D / 20.37 \times 10^9)^{-0.3509}$$

At 300B tokens: **1.15e-4** (990M), **9.8e-5** (2B), **8.6e-5** (4B). These hold for AdamW only (§7.3).

A contradiction we are obliged to report. The K3 technical report finds that with hyperparameters searched independently per schedule, **cosine outperforms WSD** on final loss. We adopt WSM regardless, for two reasons we consider sufficient but not decisive: (a) checkpoint merging is never mentioned in K3, so their comparison concerns *WSD-with-decay* and does not address WSM; (b) our selection criterion is an executable benchmark, not final

loss. The methodological point in their result — hyperparameters must be re-searched per schedule, not transferred — we accept, and it is a debt we have not yet paid.

10. Reproducibility

Everything in this report is reproducible from the repository. The architecture is additive and behind flags: with defaults, the model is **bit-identical** to v1, which is verified continuously against a frozen copy of the v1 source (`models/___old/`) — published checkpoints load and generate identically under the new code.

```
python utils/bin.arch_budget.py --tokens 300e9      # parameter and LR budget
python eval/bin.gate_arch_v2.py                    # 12 correctness gates
python eval/bin.bits_per_byte.py --build            # freeze the byte set
```

The gate suite covers: bit-identity with v1 (logits and parameter names), initialisation coverage, per-component parameter cost against the arithmetic budget, hybrid wiring, short-convolution exactness in fp64, document isolation of the recurrent state, and cache-versus-recompute agreement in generation.

Dependencies. The recurrent kernels come from `flash-linear-attention` (MIT) [14]: Triton kernels, no weights, the same relationship we have with flash-attn or cuBLAS. The version is **pinned**: kernel conventions have already changed between releases in ways that do not raise errors. Specifically, with `use_gate_in_kernel=False` the kernel expects the gate's *chunk-cumulative sum scaled by $1/\ln 2$* , not the per-position log-decay; passing the latter compiles, runs, and is wrong by 35%. We record this because it is exactly the class of silent error this report is trying to be careful about.

11. Conclusion

Skylar 2 is a modest set of architectural changes with a measured effect: **21.9% lower perplexity at +3.0% parameters and 2.45× faster training**, reproducible across three seeds on real COBOL. The two ideas we consider contributions are both small — a dimensioning rule and an algebraic identity — but each converts a technique from "too expensive to adopt" into "free", which is the difference between reading a paper and using it.

We also removed a component after measuring it, disabled another for a data-pipeline reason, deferred a third for a hyperparameter reason, and corrected three of our own previously-published numbers. We consider that part of the result rather than an embarrassment: an architecture report that contradicts none of its sources and rejects none of its candidates has not measured enough.

The open question is scale. Nothing here predicts 4B, and the next step is the 990M trained on the full 300B-token corpus — which is why we built it.

References

- [1] Zhang & Sennrich. *Root Mean Square Layer Normalization*. arXiv:1910.07467.
- [2] Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. arXiv:2104.09864.
- [3] Ainslie et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. arXiv:2305.13245.
- [4] Shazeer. *GLU Variants Improve Transformer*. arXiv:2002.05202.
- [5] Yang et al. *Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer*. arXiv:2203.03466.
- [6] Kimi Team. *Kimi Linear: An Expressive, Efficient Attention Architecture*. arXiv:2510.26692.
- [7] Moonshot AI. *Kimi K3 Technical Report*. 2026.
- [8] Kimi Team (Chen, Zhang, Su et al.). *Attention Residuals*. arXiv:2603.15031.
- [9] Qiu et al. *Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free*. arXiv:2505.06708.
- [10] DeepSeek-AI. *DeepSeek-V3 Technical Report*. arXiv:2412.19437.
- [11] Liu et al. *Muon is Scalable for LLM Training (Moonlight)*. arXiv:2502.16982.
- [12] Tian et al. *WSM: Decay-Free Learning Rate Schedule via Checkpoint Merging for LLM Pre-training*. arXiv:2507.17634.
- [13] Zhou, Xing, Huang, Qiu & Guo. *How to Set the Learning Rate for Large-Scale Pre-training?* arXiv:2601.05049.
- [14] Yang, Zhang & Li. *flash-linear-attention*. MIT licence. github.com/fla-org/flash-linear-attention.
- [15] Chowdhery et al. *PaLM: Scaling Language Modeling with Pathways*. arXiv:2204.02311.
- [16] Touvron et al. *LLaMA: Open and Efficient Foundation Language Models*. arXiv:2302.13971.
- [17] Qwen Team. *Qwen2.5 Technical Report*. arXiv:2412.15115.

References [1]-[14] were verified against their primary sources. [15]-[17] support the choice of `dropout = 0` during pre-training and the Qwen3-lineage architecture, and are cited from established literature.